Build prompt: Bolo Voice v4.4 (Android, on-device Gemma speech-to-text)

Bolo Voice · Android · On-device Gemma speech-to-text

> *Paste everything below into an AI coding assistant, or give it to an Android developer. It describes the complete, working Bolo Voice v4.4 app: what to build, the exact technology versions, every feature and its rules, the model prompts, the design, the tests, and the build problems that were already hit and solved.*

---

1. Role and goal

You are a senior Android engineer. Build **Bolo Voice v4.4**, a native Android app (Kotlin, plain Android Views, no Compose). It records speech and transcribes it **fully on the phone** with Google's **Gemma** model through the **LiteRT-LM** runtime. It has:

- live grey drafts that turn into final text at natural pauses
- strong support for Indian languages
- a second "Refine" pass after recording
- a word-safe punctuation polish
- a structured summary exported as PDF
- a dark neon "techno HUD" interface

The app must work **offline** (no INTERNET permission), need **no API key and no sign-in**, and produce a release APK that installs directly by sideloading.

Deliver a complete Gradle project that builds with `gradle assembleRelease`, passes all unit tests (`gradle testReleaseUnitTest`), and outputs a signed, installable APK.

2. Background: why this design

An earlier "v2.4 APK" was a 10 KB placeholder that redirected to a private web page, and browser speech recognition doesn't work inside an Android WebView. The fix is a **native app with an on-device model**. Don't use WebViews, cloud speech APIs, Android `SpeechRecognizer`, or Gemini cloud.

3. Target device and constraints

- Primary test phone: **Motorola ThinkPhone 25**, with a MediaTek Dimensity 7300 chip, **8 GB RAM**, and Android 14 or newer.
- `minSdk 31`, `targetSdk 36`, `compileSdk 36`. Portrait only.
- Recommended model: **Gemma 4 E2B** (`gemma-4-E2B-it.litertlm`, about 3.6 GB, Apache-2.0, no Hugging Face login needed) from `huggingface.co/litert-community/gemma-4-E2B-it-litert-lm`. **Gemma 3n E2B** (`google/gemma-3n-E2B-it-litert-lm`, which requires accepting a license) also works. Don't bundle the model in the APK.
- Don't use **E4B** models on 8 GB phones, or the `*.mediatek.*`, `*_Qualcomm_*`, `-web` or `.task` variants.

4. Exact tech stack (these versions matter)

Item	Value
Android Gradle Plugin	`com.android.application` **8.13.0**
Kotlin plugin	`org.jetbrains.kotlin.android` **2.4.0**. LiteRT-LM 0.18.1 ships Kotlin 2.4 metadata, and Kotlin 2.2 fails with "incompatible version of Kotlin" plus hundreds of unresolved references.
Gradle	**8.14.3**
JDK	**17** (`jvmToolchain(17)`)
LiteRT-LM	`com.google.ai.edge.litertlm:litertlm-android:0.18.1`
AndroidX	`activity-ktx:1.10.1`, `core-ktx:1.16.0`, `lifecycle-runtime-ktx:2.9.2`
Coroutines	`kotlinx-coroutines-android:1.10.2`
Tests	`junit:junit:4.13.2` (plain JVM unit tests)

Build settings:

- `applicationId com.bolovoice.app`, `versionCode 44`, `versionName "4.4.0"`.
- Release build: `isMinifyEnabled = false`, so the JNI bindings stay intact, and signed with the **debug** signing config so it installs directly. Add a comment saying to replace it before any Play Store release.

Root `build.gradle.kts`: declare both plugins with `apply false`. Also support a Gradle property `-PboloBuildDir=<path>` that moves `layout.buildDirectory` for all projects to `<path>/<project.name>`.
Why: building inside a OneDrive-synced folder fails with `AccessDeniedException`, because OneDrive locks freshly written build files.

Manifest:

- `RECORD_AUDIO` permission only.
- `<uses-native-library android:name="libvndksupport.so" android:required="false"/>` and the same for `libOpenCL.so`. LiteRT-LM needs these for the GPU.
- `largeHeap="true"`, `allowBackup="false"`.
- A `FileProvider` with authority `${applicationId}.files` and `cache-path` `pdf/`, for sharing PDFs.
- The single activity uses `configChanges="orientation|screenSize|screenLayout|keyboardHidden"`.

5. LiteRT-LM API usage (Kotlin)

```
val engine = Engine(EngineConfig(
    modelPath = path, backend = Backend.GPU(),   // try GPU first, fall back to Backend.CPU()
    audioBackend = Backend.CPU(), maxNumTokens = 4096, cacheDir = context.cacheDir.path))
engine.initialize()                                // slow (5–60 s): never on the UI thread
engine.createConversation(ConversationConfig(
    samplerConfig = SamplerConfig(topK = 1, topP = 1.0, temperature = 1.0))).use { conv ->
    conv.sendMessageAsync(Contents.of(Content.AudioBytes(wavBytes), Content.Text(prompt)),
                          maxOutputToken = 512).collect { msg -> /* msg.toString() = new text */ }
}
```

Rules:

- **Engine lifecycle:** if `initialize()` throws, close the half-built engine before trying the CPU. Report which backend loaded (`backend.name`).
- **One conversation per request**, to keep context small and stop text leaking between segments.

- **Greedy decoding:** `topK = 1`.
- **Streaming chunks are new text.** Append them, but tolerate a runtime that resends the whole text: if a chunk is longer than the text so far and starts with it, replace instead of appending.
- **Cancellation:** on coroutine cancellation call `conversation.cancelProcess()` and rethrow.
- **Serialise all generation with a `Mutex`.** All Gemma work (load, transcribe, summarise, polish, close) also runs on **one dedicated single-thread executor dispatcher**.
- **Audio format: 16 kHz, mono, 16-bit PCM WAV bytes**, with a 44-byte RIFF header.
- **Warm-up:** right after loading, send 1 s of a quiet 16 kHz hum with `"Transcribe this audio."` and `maxOutputTokens = 4`, so GPU kernels compile before the first real segment. A failed warm-up isn't fatal.

6. Source files to create (`com.bolovoice.app`)

`MainActivity.kt`, `ChunkedRecorder.kt`, `AudioUtils.kt`, `SessionAudio.kt`, `GemmaTranscriber.kt`, `TranscriptLanguage.kt` (also holds `enum Script`), `TextChecks.kt`, `IndicText.kt`, `ModelStore.kt`, `PdfExporter.kt`, `WaveformView.kt`, `RecordButton.kt`, `HudPanel.kt`, `TechBackgroundView.kt`.

Tests: `TextChecksTest`, `AudioUtilsTest`, `SessionAudioTest`, `IndicTextTest`.

7. Model import (`ModelStore`)

- **Picking the file:** the user picks the `.litertlm` with `ActivityResultContracts.OpenDocument` (`*/*`).
- **Checks before copying:** the name must end with `.litertlm`, the size must be over 500 MB, and free space must be more than the size plus 300 MB.
- **Copying:** stream it into `filesDir/models/gemma-3n.litertlm` with 1 MB buffers and percentage progress. Write to a `.tmp` file first and rename it only when the copy completes, so a failed import never deletes the working model. Then clear `cacheDir`, because stale GPU caches break loading.
- **On launch:** if the model file exists, load it automatically.
- **If an import fails:** if the old model still exists, show the error as a toast and reload the old model.

8. Recording and segmentation (`ChunkedRecorder`)

- **Mic setup:** `AudioRecord` with `VOICE_RECOGNITION`, 16 kHz mono PCM16, read in 100 ms frames on `Dispatchers.IO`.
- **Prefer an external mic:** look for input devices of type USB headset, then USB device, then wired headset, and call `setPreferredDevice`. Expose `micName` (`PHONE`, `USB` or `WIRED`).
- **Segmenting:** gather frames into a buffer. When it reaches **20 s**, cut at the **quietest 100 ms window** between 10 s and the buffer length, scanning in 50 ms steps (`AudioUtils.quietestCut`). Emit that segment and keep the remainder.
- **Live drafts:** `snapshot()` returns a copy of the not-yet-cut audio. Guard the buffer with a lock.
- **Stopping:** stop by cancelling the coroutine. In `finally`, flush the tail if it's 0.5 s or longer, release the recorder, and report level 0.
- **Levels:** report each frame's RMS through `onLevel`, which drives the visualisers.
- **If the mic fails to initialise:** release it and raise "Microphone is unavailable (is another app using it?)".

9. Audio clean-up (`AudioUtils`, all pure Kotlin and unit-tested)

Every segment goes through `prepareForModel` = `highPass` → `compressSilence` → `level`.

1. **Frame levels:** RMS per 100 ms frame. **Adaptive speech threshold** = 2.5 × the 20th-percentile frame RMS, clamped to [0.012, 0.08].
2. `hasSpeech`**:** at least 3 frames above the threshold. Silent and noise-only segments are **never sent** to Gemma, because they are the main source of made-up text.
3. `highPass`**:** a 2nd-order Butterworth biquad at **80 Hz** (RBJ cookbook). It removes DC offset and rumble from AC, fans, traffic and handling.
4. `compressSilence`**:** mark speech frames, then grow each one by **3 frames (300 ms)** of padding. Trim everything before the first and after the last kept frame. Gaps longer than **1 s** become **0.5 s** (keep 2 frames from the start of the gap and 3 from its end). If there's no speech, return the input unchanged.
5. `level`**:** measure RMS over the speech frames only. Gain = 0.1 ÷ speechRMS, clamped to [1, 8], and skipped if under 1.05. Apply a **soft limiter** above 0.9 (`0.9 + 0.1·tanh((|x|−0.9)/0.1)`).
6. `edgeSilence`**:** returns (leading, trailing) seconds of non-speech, used for paragraph breaks.
7. `toWav`**:** a 44-byte little-endian RIFF header plus the PCM data.

10. Session audio and Refine (`SessionAudio`)

- **Saving:** every emitted segment is also appended to `filesDir/session.pcm` (raw PCM16 little-endian, about 1.9 MB/min). Expose `seconds` and `exactSeconds`, and clear the file together with the transcript.
- **Replaying:** `forEachWindow(minSec = 20, maxSec = 28, suspend block)` streams the file back in windows cut at pauses, using the same `quietestCut`, without loading the whole file. It must reproduce the audio exactly: nothing lost or duplicated.

11. Languages (`TranscriptLanguage` and `Script`)

`enum Script`**:** LATIN, DEVANAGARI, BENGALI (Bengali-Assamese), GURMUKHI, GUJARATI, ODIA, TAMIL, TELUGU, KANNADA, MALAYALAM and ARABIC (Perso-Arabic). Each has a display name, a sample word and Unicode ranges. Latin is `41–5A, 61–7A, C0–24F`; Devanagari is `900–97F, A8E0–A8FF`; Arabic is `600–6FF, 750–77F, FB50–FDFF, FE70–FEFF`. The other scripts use their standard Unicode blocks.

`enum TranscriptLanguage`, in this order: AUTO, then the six primary (★) languages **HINDI, ENGLISH, HINGLISH, TAMIL, MALAYALAM, GUJARATI**, then BENGALI, MARATHI, TELUGU, URDU, KANNADA, ODIA, PUNJABI, ASSAMESE, NEPALI, KONKANI, SINDHI, KASHMIRI, MAITHILI, SANSKRIT. Each has a native-script label (e.g. `"★ தமிழ் · Tamil"`), an English name, a script, and a `primary` flag.

Primary-language examples and tips (used in prompts):

- **Hindi:** "कल की meeting में हम budget discuss करेंगे, ठीक है?" Tip: do not use Urdu script.
- **Tamil:** "நாளைக்கு meeting-ல budget பத்தி பேசலாம், சரியா?" Tip: write colloquial Tamil as spoken; don't convert it to formal written Tamil.
- **Malayalam:** "നാളെ meeting-ൽ budget ചർച്ച ചെയ്യാം, ശരിയല്ലേ?" Tip: use proper chillu letters (ൽ ൾ ൻ ർ ൺ).
- **Gujarati:** "કાલે meeting માં budget ની ચર્ચા કરીશું, બરાબર?" Tip: this is Gujarati, not Hindi; never Devanagari.
- **Hinglish:** Roman letters as typed in India, e.g. "Kal ki meeting mein hum budget discuss karenge, theek hai?", with common spellings (hai, nahi, kya, mein, aur, kyunki, lekin). Don't translate or use Devanagari.
- **English:** possibly an Indian accent; standard spelling; keep Indian names and places as spoken.

`instruction` **text:**

- **AUTO:** "First identify the spoken language. It is most likely Hindi, English, Hinglish (Hindi mixed with English), Tamil, Malayalam or Gujarati, but it may be another Indian language. Start your output with its English name in square brackets, for example [Tamil] or [Hinglish]. Then write the transcript in that language's own script; write Hinglish in Roman letters. Write English words in English."
- **Other languages:** "The speech is in X; it may include English words. Write X in <script> script[, for example: "<example>"]. Write English words in English letters. Do not translate. <tip>"

`strictInstruction` (used on retry): "IMPORTANT: the speaker is speaking X, not any other language. You must write in <script> script, like "<example or sample>". Never translate into English, Hindi or any other language." For Hinglish: Latin letters only, no Devanagari. For English: English only.

`summaryName`**:** English → "English"; AUTO → "the same language and script as the transcript"; Hinglish → "Hinglish (Hindi and English, written in Roman script)"; otherwise "X, in <script> script".

`fromEnglishName(name)`**:** case-insensitive lookup by English name.

12. Transcription logic (`GemmaTranscriber`)

Transcription prompt (in this order):

1. If there's context: `For context, the transcript just before this audio ends with: "<last ~160 chars, timestamps stripped>". Do not repeat that text; transcribe only this audio.`
2. If there are terms: `Names and terms that may be spoken (spell them exactly like this when you hear them): <terms>.`
3. `Transcribe this audio verbatim. <instruction>[ <strictInstruction>] Output only the spoken words with normal punctuation: no title, labels, timestamps, translation or commentary. If there is no intelligible speech, output nothing.`

Per segment:

1. **Generate** with `maxOutputToken = 512`, and audio before text in the message.
2. **Clean the output:** strip a `[Language]` tag (and return "" for a half-streamed tag), the `Transcription:` / `Transcript:` prefixes and quotes. Treat "no speech", "[silence]" and similar as empty. Run `collapseRepeats`.
3. **Script check:** if a language is selected and `scriptScore(text, script) < 0.7`, **retry once with the strict instruction** (finals only, not drafts) and keep the higher-scoring result.
4. **Auto-detect:** `detected = resolveLanguage(tagLanguage, text)`.
5. **Normalise:** apply `IndicText.normalize(text, language or detected)`.

`summarize`**:**

- **Long transcripts:** over 3000 characters, map-reduce into bullet notes (at most 3 rounds, splitting at sentence ends).
- **Final summary:** write it in the chosen language with exactly four English headings: OVERVIEW (2–3 sentences), KEY POINTS, DECISIONS ("- None" if empty) and ACTION ITEMS (with owners; "- None" if empty).
- **Formatting:** plain text; strip `**` and leading `#`.
- **Progress:** report it, e.g. "reading part i of n", then "writing summary · N words".

`polish(text, language)`**:**

- **Paragraphs:** split on blank lines, and keep each paragraph's leading `[mm:ss]` stamp untouched.
- **Pieces:** split each paragraph body into pieces of at most about 1200 characters at sentence ends.

- **Prompt:** "Fix only the punctuation, capitalisation and spacing of the text below. Keep every word exactly as it is: do not translate, correct, add, remove or reorder words, and keep the same script. Output only the corrected text."
- **Safety check:** accept the result **only if** `TextChecks.wordsPreserved(original, fixed)`; otherwise keep the original. Normalise either way, then rebuild the paragraphs and their stamps.

13. Text checks (`TextChecks` and `IndicText`, pure and unit-tested)

`TextChecks`:

- `scriptScore(text, expected)`: count letters by script. Under 4 letters → 1.0. For Latin, the score is the Latin share. Otherwise: no native letters → 0; else `nativeShare × min(1, overallShare/0.3)`, where nativeShare = native ÷ (native + other non-Latin). English words inside Indian-language text are allowed.
- `looksHinglish`: 2 or more hits and at least 12% of words are from a set of about 55 common romanised Hindi words (hai, nahi, kya, mein, aur, ke, ki, ko, toh, bhi, kar, raha, tha, yeh, woh, haan, accha, matlab, kyunki, lekin, abhi, kal, aaj, theek, bahut, hum, aap, mujhe, chahiye, …).
- `guessLanguage`: the dominant script decides, Devanagari maps to Hindi, and Latin is split into English or Hinglish using `looksHinglish`. Requires at least 6 letters; a native script needs at least a 30% share, Latin at least 80%.
- `resolveLanguage(tagged, text)`: the script wins when it disagrees with the tag; for Latin, the vocabulary check decides; otherwise trust the tag (e.g. Marathi vs Hindi).
- `collapseRepeats`: collapse a phrase of 1–6 words repeated 3 or more times in a row into one copy.
- `removeOverlap(prev, next)`: drop up to 15 leading words of `next` that repeat the end of `prev`. Compare lowercased, trimming only punctuation, so Indic vowel signs and viramas are kept.
- `wordsPreserved(a, b, 0.97)`: reject if `b` has more than 5% (+1) extra words; otherwise the word-level longest common subsequence must be at least 97% of `a`'s words.
- `tail`: the last about 160 characters at a word boundary, with timestamps stripped.

`IndicText.normalize(text, language)`:

- **Encoding:** NFC normalisation, and Malayalam chillu repair (ൺ ൻ ർ ൽ ൾ ൿ → ണ ന ര ല ള ക).
- **Joiners:** remove ZWNJ/ZWJ at word edges only.
- **Spacing:** no space before `, ! ? ; : । ॥`, and one space after them when a letter follows.
- **Danda:** for Hindi, Marathi, Nepali, Sanskrit, Maithili, Konkani, Bengali and Assamese, turn "." after a Devanagari or Bengali letter into "।". Collapse repeated "।" and double spaces.
- **Never change** "3.5", Tamil, Malayalam, Gujarati or English full stops.

Timestamps and paragraphs:

- **Formats:** `[mm:ss]`, or `[h:mm:ss]` past an hour.
- `appendSegment(transcript, segment, speechStartSec, gapBeforeSec, newSession)`: the first segment gets a stamp. A gap of 2.0 s or more, or a new recording, starts `\n\n[mm:ss]` and a new paragraph. Otherwise append with a space, and ignore empty segments.
- **Helpers:** `stripTimestamps` and `timestamps`.

14. The live pipeline (`MainActivity`)

- **Segment queue:** each emitted segment becomes a `Chunk(pcm, startSec = sessionAudio.exactSeconds before appending, newSession = first chunk of this recording)` in an unlimited `Channel`.
- **Worker coroutine (main thread):**
  - **Finished chunks always first.** Transcribe with `final = true`.
  - **Paragraph gaps:** gap = previous trailing silence + this chunk's leading silence. `commit(text, startSec + lead, gap, newSession)` goes through `appendSegment`, after `removeOverlap`.
  - **Live drafts:** when the queue is empty, recording is on and LIVE is enabled, transcribe the recorder `snapshot()` once at least 1.5 s exists and 3 s or more of new audio has arrived. Drafts are **not streamed**: they replace the grey text only once complete, so it never shrinks and regrows.
  - **Streaming finals:** final text streams into the grey area, but replaces the draft only when at least as long. A generation counter discards stale updates.
  - **Leftover drafts:** when recording stops and nothing is pending, drop a leftover draft.
- **Auto-detect lock:** in AUTO mode, lock onto a language once **two final segments in a row** agree. From then on, transcribe with that language explicitly. Changing the language or clearing resets the lock.
- **Stats line:** `ENGINE · MIC X · LANG · LAT n.ns · SPD n.n×`. LAT is the time to transcribe the last final segment; SPD is audio length divided by that time.
- **Refine (⟳ REFINE):**
  - **When available:** not recording, nothing pending, model loaded, and at least 2 s recorded. While running, the button becomes ■ CANCEL.
  - **What it does:** walk `forEachWindow(20, 28)`, track each window's offset, skip non-speech windows (counting them as gap), transcribe with `allowRetry = true`, context = tail of the refined text, and the locked or detected language. Build timestamped paragraphs, then `polish`.
  - **Result:** show a dialog with the new and old word counts and a 600-character preview, with **Replace transcript** and **Keep original**.
- **Summary (✦ SUMMARISE → PDF):** blocked while recording or pending, with a toast. Ask for English or the spoken language (skip the question if the spoken language is English). Write the PDF to `cache/pdf/BoloVoice-Summary-yyyy-MM-dd-HHmm.pdf`. Show the summary dialog with Save PDF (`CreateDocument`), Share PDF (`FileProvider`) and Close.
- ✎ **TERMS:** a dialog with a multi-line field of comma-separated names and jargon (at most 40, de-duplicated), saved in prefs. The button shows the count (✎ `TERMS·5`).
- **Persistence:** save the transcript, language, live toggle, terms, Roman view, font size and recorded time in `SharedPreferences`. On Clear, also clear the session audio.
- **Mic permission:** request it at runtime; keep the screen on while recording.
- **Keep it responsive:** never block the UI thread.

15. PDF (`PdfExporter`)

- **Page setup:** A4 (595 × 842 pt) `PdfDocument`, 50 pt margins. Use `StaticLayout`, drawn **line by line** so paragraphs flow across pages; this handles all Indic scripts and right-to-left text.
- **Content:** title "Bolo Voice — Meeting Summary"; a metadata line (date, recorded length, "Transcribed on-device with Gemma"); the four headings in indigo; bullets as "•"; then "Full transcript".
- **Footer:** "Page N · Generated on-device by Bolo Voice".

16. Design: dark neon "techno HUD"

Palette:

- Background `#05070F`; panel `#C70B1020` (slightly see-through); solid panel `#0B1020`; border `#1E2A44`.
- Text `#E6F1FF`; muted `#7C8BA6`.
- Accents: cyan `#22D3EE`, violet `#8B5CF6`, pink `#E879F9`.
- Status colours: ok `#34D399`, warn `#FBBF24`, error `#F87171`.

Drawables:

- **`bg_app`:** near-black with two radial glows.
- **`panel`:** a 1 dp cyan → violet gradient border around the glass fill.
- **`pill`:** a border-coloured outline.
- **`btn_primary`:** a ripple over a cyan → violet → pink gradient, dark when disabled.
- **`btn_ghost`:** a gradient border around a dark fill.
- **`chip_selected`:** a 1.5 dp tri-colour gradient border around `#0E1A33`.
- **`dot`.**
- Switch thumb and track colour lists.

Theme: `Theme.Material.NoActionBar` with a transparent status bar, and a dark dialog theme using the panel background.

Screen, top to bottom (root `FrameLayout` with `TechBackgroundView` behind a vertical `LinearLayout`, `fitsSystemWindows`, 16 dp side padding):

1. **Header:** "BOLO VOICE" (26 sp, condensed bold, 0.14 letter spacing) with a cyan → violet → pink gradient shader and a cyan glow (`setShadowLayer`). Below it, "ON-DEVICE AI · GEMMA · OFFLINE" in a monospace label.
2. **Status panel:**
   - a coloured dot: grey when idle, amber while busy, green when ready, red on error;
   - the status text in monospace, e.g. "GEMMA READY · GPU · OFFLINE", or "Warming up the engine…" after load;
   - the **stats line** in violet, 9.5 sp;
   - a compact ghost button: **Load model** / **Change**.
3. **Language chips** (`HorizontalScrollView`): `AUTO · HI · EN · HING · TA · ML · GU · ⋯`. The selected chip uses `chip_selected` with cyan text. `⋯` opens a list of the other 14 languages and then shows the choice (`⋯ BENGALI`). Chips are disabled while recording.
4. **Recorder `HudPanel`:**
   - **`RecordButton`** (108 dp), drawn by hand: a dark radial core and a sweep-gradient ring with a white mic icon. While recording, the ring spins, the core turns pink → violet with a stop icon, and two halos pulse outward.
   - **Radial visualiser:** 48 sweep-gradient bars around the ring that grow with the voice level (`pushLevel`).
   - **Right column:** a 30 sp monospace timer, a 34 dp `WaveformView` (40 rounded gradient bars scrolling left and fading with age, level = √rms × 2.2), and a row with ✎ **TERMS** and a **LIVE** switch.
5. **Status line** (monospace 11 sp, cyan), for example "◆ Tamil · auto-locked · ▸ listening · live draft on ▌". The cursor ▌ blinks every 500 ms while recording.
6. **Transcript `HudPanel`** (fills the remaining height):

   - **Header:** "▌TRANSCRIPT", the word count (timestamps excluded), and three header chips: **अ→A** (Roman view), **A−** and **A+**.
   - **Body:** a `ScrollView` with fading edges and an **editable-buffer** `TextView` (16 sp by default, 13–28, line spacing 1.35).
   - **Final text** is white, the **live draft** grey italic, and **timestamps** violet.
7. **Actions:** ⟳ **REFINE** (ghost, weight 1) beside ✦ **SUMMARISE → PDF** (primary, weight 1.6), then ⧉ **Copy** · ↗ **Share** · ✕ **Clear**.

Custom views:

- `TechBackgroundView`:
  - a faint cyan grid with 28 dp spacing, drifting down about 6 dp/s and fading toward the bottom;
  - a synthwave perspective floor of violet lines below 72% of the height;
  - two wandering radial glows;
  - redraws about every 33 ms and stops whenever it isn't visible.
- `HudPanel`: a `LinearLayout` that draws cyan and violet corner brackets (14 dp, 2 dp stroke). While `scanning` (recording or refining), a neon scan line with a soft trail sweeps down every 2.4 s.

Smooth transcript rendering (this fixed real flicker bugs):

- **Diff instead of replace:** keep one `Editable` buffer. Find the common prefix with the target text and `replace` only the changed tail; never `setText` the whole transcript.
- **Throttle:** coalesce streamed updates to about 12 redraws per second.
- **Scrolling:** follow the newest text only if the user is within 64 dp of the bottom, using `scrollTo(0, child.bottom)`. Never use `fullScroll(FOCUS_DOWN)`.
- **Not selectable:** the TextView isn't selectable; use the Copy button.
- **Timestamp styling:** restyle with a dedicated `TimestampSpan` class in the changed region only, removing old ones first.

Roman view:

- **What it does:** shows the transcript through `android.icu.text.Transliterator.getInstance("Any-Latin; Latin-ASCII")` (नमस्ते → namaste).
- **Caching:** cache the transliterated transcript and redraw everything when toggled.
- **What follows it:** Copy and Share use what's on screen; the PDF and summary always use the original script.

17. Unit tests (all must pass; 38 in total)

- **AudioUtilsTest:**
  - silence and steady noise are not speech;
  - speech over noise is detected, and quiet speech in a quiet room is detected;
  - the high-pass cuts 30 Hz rumble by more than 80% and keeps 400 Hz above 90%;
  - silence trimming turns 12 s (3 + 1 + 4 + 1 + 3) into 2.5–4 s; all-silence is unchanged;
  - quiet speech is boosted more than 4× without clipping, and loud speech isn't boosted;
  - the full chain produces a valid WAV.
- **TextChecksTest:**
  - native scripts match, and English words inside them are allowed;
  - translation into English is rejected, and the wrong Indic script is rejected;

  - Hinglish must be Latin, and short text isn't judged;
  - repetition loops collapse, echoed context is removed, and the tail cuts at a word;
  - the four primary scripts are identified, and Hinglish is told apart from English;
  - the script overrides a wrong tag, and the same script keeps the finer tag (Marathi);
  - punctuation-only polish is accepted, but polish that translates, drops or adds words is rejected;
  - sentence splitting is bounded and lossless.
- **SessionAudioTest:** 70 s replays as 3–4 windows of 20–28 s, bit-exact; cuts land in pauses.
- **IndicTextTest:**
  - Malayalam chillu repair; stray joiners removed while inner joiners are kept; Tamil composition to NFC;
  - Hindi danda; Tamil, English and "3.5" untouched; punctuation spacing fixed;
  - timestamp formats; paragraphs on 2 s or longer gaps; empty segments ignored; timestamps stripped from the tail.

18. Known pitfalls (already solved; avoid repeating them)

1. **Kotlin version:** Kotlin must be **2.4.0** for LiteRT-LM 0.18.1. See §4.
2. **OneDrive builds:** building inside OneDrive fails with `AccessDeniedException`. Build with `-PboloBuildDir=<folder outside OneDrive>`.
3. **JUnit return type:** a test written as `fun x() = runBlocking { … }` must declare `: Unit`, or JUnit reports `InvalidTestClassError`.
4. **Suspend calls in lambdas:** you can't call a suspend function inside `joinToString { }` or `map { }` lambdas; use `for` loops.
5. **Self-referencing locals:** don't write a local `val x = x` that shadows a property; give it a different name.
6. **Flicker and jumping:** caused by full `setText`, by drafts that stream from empty, and by `fullScroll` on a selectable TextView. See §16.
7. **Wrong-language output:** caused by Auto-detect re-guessing every segment, and by no script verification. See §12–14.

19. Build and verify

```
gradle testReleaseUnitTest assembleRelease --no-daemon "-PboloBuildDir=C:\path\outside\onedrive\out"
apksigner verify app-release.apk
aapt2 dump badging app-release.apk   # expect package com.bolovoice.app, versionCode 44, versionName 4.4.0
```

Copy the APK out as `BoloVoice-v4.4.apk` (about 55 MB, with `lib/arm64-v8a/liblitertlm_jni.so` inside).

20. Acceptance checklist (on the phone)

1. **Install:** it installs over older versions. The first launch asks for the model; importing shows a percentage, then "Warming up the engine…", then "GEMMA READY · GPU · OFFLINE" (or CPU).
2. **Recording:** tapping the mic starts the spinning ring, halos, radial bars, waveform, timer, blinking cursor and scan lines.
3. **Live text:** grey drafts appear within a few seconds and become white text at pauses. Long pauses create `[mm:ss]` paragraphs. No flicker; scrolling up stays put.
4. **Language chips:** HI, TA, ML, GU, HING and EN each produce the correct script, with English words left in English. AUTO shows "hearing X, confirming…", then "◆ X · auto-locked".

5. **Script details:** Hindi uses "।", and Malayalam uses atomic chillu letters.
6. **Refine:** after Stop, ⟳ REFINE shows progress, then offers Replace or Keep. Paragraphs and stamps survive, and words never change during the polish.
7. **Summary:** ✦ SUMMARISE → PDF asks for a language and produces a four-section summary; Save and Share both work.
8. **Roman view:** अ→A switches to Roman letters and back. A−/A+ resize the text.
9. **Terms:** names added to ✎ TERMS appear spelled correctly.
10. **External mic:** plugging in a USB-C mic shows "MIC USB" on the stats line.
11. **Offline:** everything works in flight mode.